

Rust逆向之引用

由于Rust具有变量移动这个特性，就让开发人员需要经常使用到引用。这样可以在不移动变量的情况下，还可以使用这个变量。

1.不可变引用

引用默认也是不可变的，下面的Rust的代码展现了引用的基本使用。此时，变量t1是对变量t进行引用。所以变量x不会移动，后续对变量t的使用就不会有问题。

```
fn test() {  
    struct Test{  
        x: i32,  
        y: i32,  
    }  
  
    let t = Test {  
        x: 1,  
        y: 2,  
    };  
  
    let t1 = &t;  
  
    let x = t.x;  
    let y = t1.y;  
}
```

对于汇编代码如下，这里可以看的出来，Rust中的引用和C/C++一样，都是将相应变量的地址赋值过去。之后，通过变量t1获取值，也是拿变量t的地址获取，这里应该是Rust编译器做了优化。

.text:0000000140001150 demo__test	proc near
.text:0000000140001150	
.text:0000000140001150	sub rsp, 18h
.text:0000000140001154	mov dword ptr [rsp], 1 ; 为变量t赋值
.text:000000014000115B	mov dword ptr [rsp+4], 2
.text:0000000140001163	mov rax, rsp ; rax=变量t的地址
.text:0000000140001166	mov [rsp+8], rax ; t1=变量t的地址
.text:000000014000116B	mov eax, [rsp] ; eax=t.x
.text:000000014000116E	mov [rsp+10h], eax ; x=t.x
.text:0000000140001172	mov eax, [rsp+4] ; eax=t1.y
.text:0000000140001176	mov [rsp+14h], eax ; y=t1.y
.text:000000014000117A	add rsp, 18h
.text:000000014000117E	retn
.text:000000014000117E demo__test	endp

2.可变引用

和定义可变变量一样，可变引用也要用到mut关键字。下面是一个简单的例子，定义一个可变引用的时候，需要被引用变量也是可变的。

```
fn test1() {  
    struct Test{  
        x: i32,  
        y: i32,  
    }  
  
    let mut t = Test {  
        x: 1,  
        y: 2,  
    };  
  
    {  
        let t1 = &mut t;  
  
        t1.x = 3;  
    }  
  
    let x = t.x;  
}
```

这里变量t1要放在一个较小的作用域内，是因为Rust编译器会为了防止数据竞争进行检查，不允许多个可变变量，在同一作用域内指向同一块内存。所以在同一作用域内，定义可变引用的时候，Rust编译器会作为以下限制：

- 不能对一个可变变量，定义两个及以上的可变引用
- 不能对一个可变变量，同时定义可变引用和不可变引用
- 如果对一个可变变量定义了可变引用，后续就不能继续使用这个可变变量

对应的反汇编代码如下，这里可以看到，可变引用和不可变引用都是编译器层面的检查。在二进制层面，它们的保存方式是一样的。

```

.text:0000000140001180 demo__test1      proc near
.text:0000000140001180
.text:0000000140001180                sub     rsp, 18h
.text:0000000140001184                mov     dword ptr [rsp], 1 ; 为变量t赋值
.text:000000014000118B                mov     dword ptr [rsp+4], 2
.text:0000000140001193                mov     rax, rsp          ; rax=变量t地址
.text:0000000140001196                mov     [rsp+8], rax      ; t1=变量t地址
.text:000000014000119B                mov     dword ptr [rsp], 3 ; t1.x=3
.text:00000001400011A2                mov     eax, [rsp]        ; eax=t.x
.text:00000001400011A5                mov     [rsp+14h], eax   ; x=t.x
.text:00000001400011A9                add     rsp, 18h
.text:00000001400011AD                retn
.text:00000001400011AD demo__test1      endp

```

3.借用

当引用类型作为函数参数传递的时候，就称其为借用。以下是一个借用的例子：

```

struct Test{
    x: i32,
    y: i32,
}

fn test() {
    let t = Test {
        x: 1,
        y: 2,
    };

    func(&t);
}

fn func(t: &Test) {
    let t1 = t;

    let x = t1.x;
    let y = t.y;
}

```

在test函数中，会将变量t的地址作为第一个参数来调用func函数：

```

.text:0000000140001150 demo__test      proc near
.text:0000000140001150
.text:0000000140001150                sub     rsp, 28h
.text:0000000140001154                mov     dword ptr [rsp+20h], 1 ; 为变量t赋值
.text:000000014000115C                mov     dword ptr [rsp+24h], 2
.text:0000000140001164                lea     rcx, [rsp+20h] ; rcx=变量t地址
.text:0000000140001169                call    demo__func
.text:000000014000116E                nop
.text:000000014000116F                add     rsp, 28h
.text:0000000140001173                retn
.text:0000000140001173 demo__test      endp

```

func函数会将变量t的地址保存起来，然后通过利用变量t的地址进行赋值：

```

.text:0000000140001180 ; void __fastcall demo::func(demo::Test *)
.text:0000000140001180 demo__func      proc near
.text:0000000140001180
.text:0000000140001180                sub     rsp, 10h
.text:0000000140001184                mov     [rsp], rcx ; 保存变量t地址
.text:0000000140001188                mov     eax, [rcx] ; eax=t1.x
.text:000000014000118A                mov     [rsp+8], eax ; x=t1.x
.text:000000014000118E                mov     eax, [rcx+4] ; eax=t.y
.text:0000000140001191                mov     [rsp+0Ch], eax ; y=t.y
.text:0000000140001195                add     rsp, 10h
.text:0000000140001199                retn
.text:0000000140001199 demo__func      endp

```

同样可以通过mut关键字来进行可变借用，以下是一个简单的例子：

```

fn test1() {
    let mut t = Test {
        x: 1,
        y: 2,
    };

    func1(&mut t);
}

fn func1(t: &mut Test) {
    t.y = 3;
    let y = t.y;
}

```

test1反汇编代码如下，和上面test反汇编没有区别：

```

.text:000000001400011A0 demo__test1      proc near
.text:000000001400011A0
.text:000000001400011A0                sub     rsp, 28h
.text:000000001400011A4                mov     dword ptr [rsp+20h], 1
.text:000000001400011AC                mov     dword ptr [rsp+24h], 2
.text:000000001400011B4                lea     rcx, [rsp+20h] ; demo::Test *
.text:000000001400011B9                call    demo__func1
.text:000000001400011BE                nop
.text:000000001400011BF                add     rsp, 28h
.text:000000001400011C3                retn
.text:000000001400011C3 demo__test1      endp

```

func1的反汇编如下，这里没什么特别的：

```

.text:000000001400011D0 ; void __fastcall demo::func1(demo::Test *)
.text:000000001400011D0 demo__func1      proc near
.text:000000001400011D0
.text:000000001400011D0                sub     rsp, 10h
.text:000000001400011D4                mov     [rsp], rcx ; 保存变量t地址
.text:000000001400011D8                mov     dword ptr [rcx+4], 3 ; t.y=3
.text:000000001400011DF                mov     eax, [rcx+4] ; eax=t.y
.text:000000001400011E2                mov     [rsp+0Ch], eax ; y=t.y
.text:000000001400011E6                add     rsp, 10h
.text:000000001400011EA                retn
.text:000000001400011EA demo__func1      endp

```

4.切片

切片是引用的一种特殊方式，用于获取一个集合中，某一段连续的元素序列。切片比较常用于字符串，下面是一个获取字符串切片的例子：

```

fn test() {
    let s = "Hello";
    let s1 = &s[1..3];
}

```

对应的反汇编如下，可以看到，在获取字符串切片的时候，会调用_ZN4core3str6traits66_LTimplu20core__ops__index_IndexLTl\$GT\$u20foru20strGT5index17h7952fc85f19b804eE函数。调用这个函数的时候，前两个参数分别是字符串的地址和长度，后两个参数分别是要获取上下标。随后，会通过该函数的返回值来赋值变量s1。

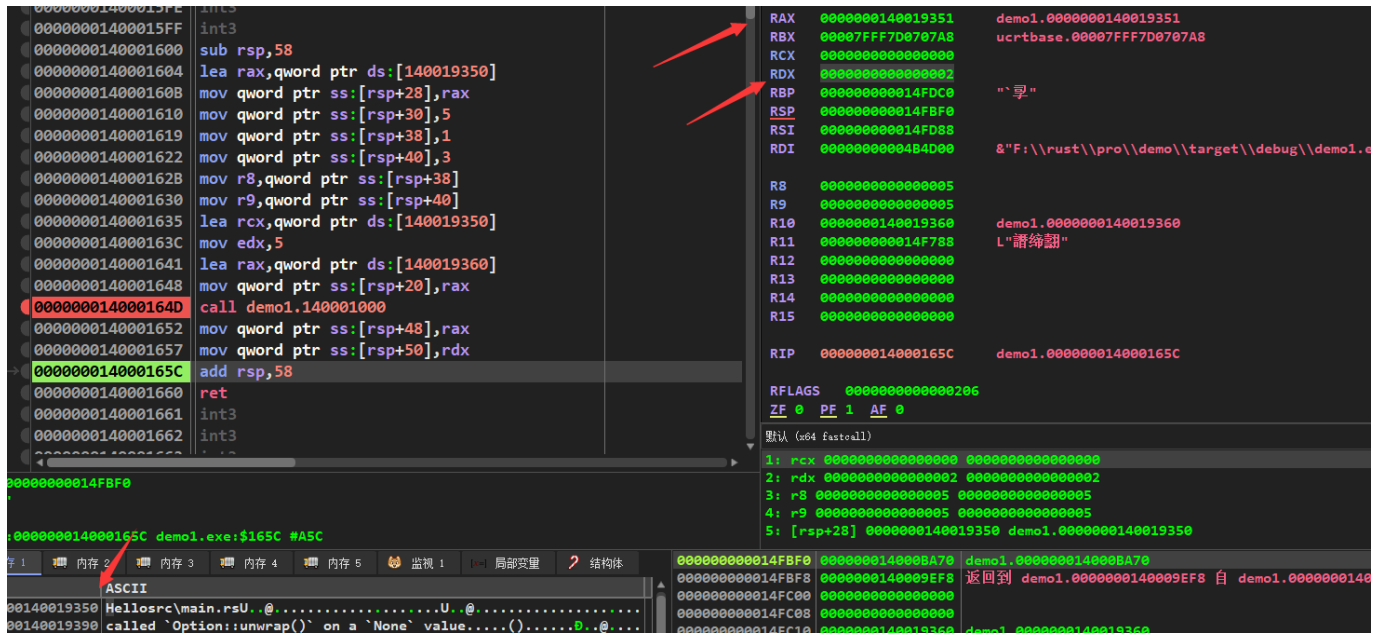
```

.text:0000000140001600 demo__test      proc near
.text:0000000140001600
.text:0000000140001600
.text:0000000140001604
"HelloSrc\\main.rs"
.text:000000014000160B      mov     [rsp+28h], rax ; 初始化变量s
.text:0000000140001610      mov     qword ptr [rsp+30h], 5
.text:0000000140001619      mov     qword ptr [rsp+38h], 1
.text:0000000140001622      mov     qword ptr [rsp+40h], 3
.text:000000014000162B      mov     r8, [rsp+38h] ; r8=1
.text:0000000140001630      mov     r9, [rsp+40h] ; r9=3
.text:0000000140001635      lea     rcx, aHelloSrcMainRs ;
"HelloSrc\\main.rs"
.text:000000014000163C      mov     edx, 5
.text:0000000140001641      lea     rax, unk_140019360 ;
rax=aHelloSrcMainRs字符串结尾的地址
.text:0000000140001648      mov     [rsp+20h], rax
.text:000000014000164D      call    _ZN4core3str6traits66_$LT$impl$u20$core__ops__index__Index$LT$I$GT$$u20$for$u20$str$GT$
$5index17h7952fc85f19b804eE ;
core::str::traits::_$LT$impl$u20$core..ops..index..Index$LT$I$GT$$u20$for$u20$str$GT$:
:index::h7952fc85f19b804e
.text:0000000140001652      mov     [rsp+48h], rax ; 用返回值初始化s1
.text:0000000140001657      mov     [rsp+50h], rdx
.text:000000014000165C      add     rsp, 58h
.text:0000000140001660      retn
.text:0000000140001660 demo__test      endp

```

`ZN4core3str6traits66LTimplu20core__ops__index__IndexLTIGT`

`u20foru20strGT5index17h7952fc85f19b804eE`函数的返回值`rax`和`rdx`，可以通过动态调试查看。从下图可以看到，此时`rax`为`0x140019351`，而`0x140019350`保存的就是字符串"Hello"。所以，`rax`返回的就是字符串切片以后的起始地址，而`rdx`保存的就是该字符串的长度。



切片也常用于数组类型，下面就是一个例子：

```
fn test1() {  
    let x = [1, 2, 3, 4, 5];  
    let y = &x[1..3];  
}
```

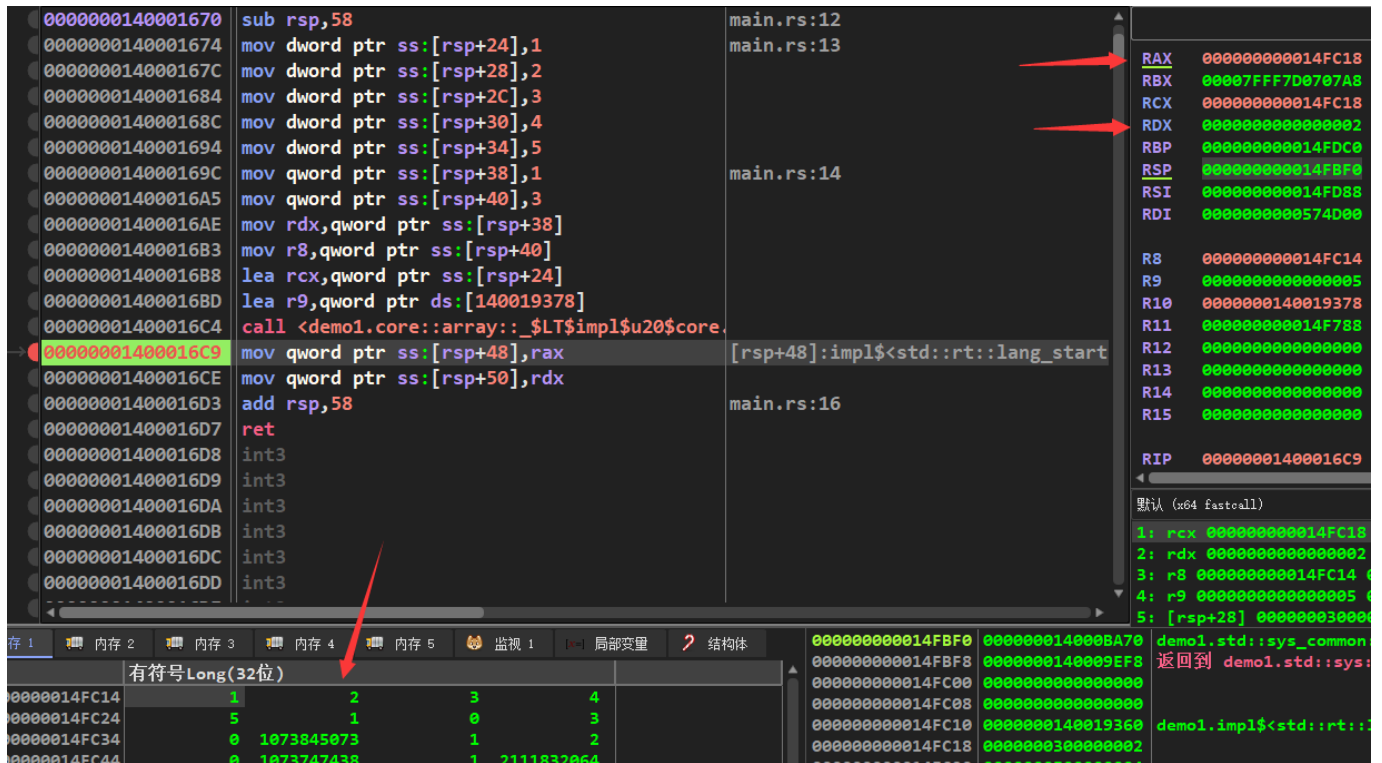
对应的反汇编如下，和字符串切片类似。核心也是调用 `ZN4core5array8LTimplu20core__ops__index__IndexLTIGTtu20foru20u5bTu3bu20Nu5dGT5index17h2a8901c981aec927E` 函数，然后用该函数的返回值给变量 `y` 赋值。

```

.text:0000000140001670 demo__test1      proc near
.text:0000000140001670
.text:0000000140001670                sub     rsp, 58h
.text:0000000140001674                mov     dword ptr [rsp+24h], 1 ; 初始化数组x
.text:000000014000167C                mov     dword ptr [rsp+28h], 2
.text:0000000140001684                mov     dword ptr [rsp+2Ch], 3
.text:000000014000168C                mov     dword ptr [rsp+30h], 4
.text:0000000140001694                mov     dword ptr [rsp+34h], 5
.text:000000014000169C                mov     qword ptr [rsp+38h], 1 ; 保存切片起始下
标
.text:00000001400016A5                mov     qword ptr [rsp+40h], 3 ; 保存切片终点下
标
.text:00000001400016AE                mov     rdx, [rsp+38h] ; rdx=1
.text:00000001400016B3                mov     r8, [rsp+40h] ; r8=3
.text:00000001400016B8                lea     rcx, [rsp+24h] ; rcx=数组首地址
.text:00000001400016BD                lea     r9, unk_140019378
.text:00000001400016C4                call   _ZN4core5array85_$LT$impl$u20$core__ops__index__Index$LT$I$GT$$u20$for$u20$$u5b$T$u3b$
$u20$N$u5d$$GT$5index17h2a8901c981aec927E ;
core::array::_$LT$impl$u20$core..ops..index..Index$LT$I$GT$$u20$for$u20$$u5b$T$u3b$$u2
0$N$u5d$$GT$::index::h2a8901c981aec927
.text:00000001400016C9                mov     [rsp+48h], rax ; 将返回值赋值给y
.text:00000001400016CE                mov     [rsp+50h], rdx
.text:00000001400016D3                add     rsp, 58h
.text:00000001400016D7                retn
.text:00000001400016D7 demo__test1      endp

```

动态调试调试如下图，可以看到这里的rax就是数组中第一个元素的地址，rdx也是切片数组的长度。



5.解引用

&符号用来对变量进行引用，其实就是获取该变量的地址。对应的，从该变量地址获取该变量值的符号*，就是解引用。下面是一个解引用的例子：

```

fn test_i32() {
    let x = 5;
    func_i32(&x);
}

fn func_i32(x: &i32) {
    let y = x;
    let z = *x;
}

```

test_i32函数，会将x变量的地址作为第一个参数来调用func_i32：

```

.text:0000000140001900 demo__test_i32 proc near
.text:0000000140001900             sub     rsp, 28h
.text:0000000140001904             mov     dword ptr [rsp+20h], 5
.text:000000014000190C             mov     dword ptr [rsp+24h], 5
.text:0000000140001914             lea     rcx, [rsp+24h] ; int *
.text:0000000140001919             call    demo__func_i32
.text:000000014000191E             nop
.text:000000014000191F             add     rsp, 28h
.text:0000000140001923             retn
.text:0000000140001923 demo__test_i32 endp

```

在func_i32中，对y变量的赋值，是直接用x变量的地址。而对变量z的赋值，则是根据变量xd的地址，从中获取变量x的值。

```

.text:0000000140001930 ; void __fastcall demo::func_i32(int *)
.text:0000000140001930 demo__func_i32 proc near
.text:0000000140001930
.text:0000000140001930             sub     rsp, 10h
.text:0000000140001934             mov     [rsp], rcx ; y=x
.text:0000000140001938             mov     eax, [rcx] ; eax=*x
.text:000000014000193A             mov     [rsp+0Ch], eax ; z=*x
.text:000000014000193E             add     rsp, 10h
.text:0000000140001942             retn
.text:0000000140001942 demo__func_i32 endp

```

在将引用作为参数调用函数的时候，会发生隐式解引用转换。

解引用转换（deref coercion）是Rust为函数和方法的参数提供的一种便捷特性。当某个类型T实现了Deref trait时，它能够将T的引用转换为T经过Deref操作后生成的引用。当我们将某个特定类型的值引用作为参数传递给函数或方法，但传入的类型与参数类型不一致时，解引用转换就会自动发生。编译器会插入一系列的deref方法调用来将我们提供的类型转换为参数所需的类型。

隐式解引用常用于&String和&str之间互相转换，String是一个用来保存了字符串的类。用这个类创建的字符串，会保存在堆内存中。下面是两种常见的，创建String变量的方法。

```

fn test_string() {
    let s = String::from("Hello");
    let s1 = "Hello".to_string();
}

```

对应的汇编代码如下，可以看到这两个函数。分别通过调用_ZN76_LTalloc_string_Stringu20asu20core_convert_From\$LT\$RFstr\$GT\$GT4from17hcf90cab7229f4c30E和_ZN47_LTstru20

as_u20alloc__string__ToString*GT*9to_string17h175d5714a9a1957fE这两个函数，来申请内存。这两个函数的第一个参数，分别是变量s和变量s1的内存地址。

```

.text:0000000140001900 demo__test_string proc near
.text:0000000140001900             push    rbp
.text:0000000140001901             sub     rsp, 70h
.text:0000000140001905             lea     rbp, [rsp+70h]
.text:000000014000190A             mov     [rbp+var_8], 0FFFFFFFFFFFFFFFh
.text:0000000140001912             lea     rdx, aAttemptToDivid+19h ; "Hellocalled
`Option::unwrap()` on a `No"...
.text:0000000140001919             mov     [rbp-48h], rdx
.text:000000014000191D             lea     rcx, [rbp-38h] ; 将变量s的地址赋值给rcx
.text:0000000140001921             mov     r8d, 5
.text:0000000140001927             mov     [rbp+var_40], r8
.text:000000014000192B             call
_ZN7_$LT$alloc__string__String$u20$as$u20$core__convert__From$LT$$RF$str$GT$$GT$4from
17hcf90cab7229f4c30E ;
_$LT$alloc..string..String$u20$as$u20$core..convert..From$LT$$RF$str$GT$$GT$::from::hc
f90cab7229f4c30
.text:0000000140001930             mov     rdx, [rbp-48h]
.text:0000000140001934             mov     r8, [rbp-40h]
.text:0000000140001938             loc_140001938: ; DATA XREF:
.rdata:000000014001F1EC↓o
.text:0000000140001938 ; try {
.text:0000000140001938             lea     rcx, [rbp-20h] ; 将变量s1的地址赋值给
ecx
.text:000000014000193C             call
_ZN47_$LT$str$u20$as$u20$alloc__string__ToString$GT$9to_string17h175d5714a9a1957fE ;
_$LT$str$u20$as$u20$alloc..string..ToString$GT$::to_string::h175d5714a9a1957f
.text:0000000140001941             jmp     short $+2
.text:0000000140001943 ; -----
-----
.text:0000000140001943
.text:0000000140001943 loc_140001943: ; CODE XREF:
demo__test_string+41↑j
.text:0000000140001943             lea     rcx, [rbp-20h] ; alloc::string::String
*
.text:0000000140001947             call
_ZN4core3ptr42drop_in_place$LT$alloc__string__String$GT$17h14850fc5b309e955E ;
core::ptr::drop_in_place$LT$alloc..string..String$GT$::h14850fc5b309e955
.text:0000000140001947 ; } // starts at 140001938
.text:000000014000194C
.text:000000014000194C loc_14000194C: ; DATA XREF:
.rdata:000000014001F1F4↓o
.text:000000014000194C             jmp     short $+2
.text:000000014000194E ; -----
-----
.text:000000014000194E
.text:000000014000194E loc_14000194E: ; CODE XREF:
demo__test_string:loc_14000194C↑j
.text:000000014000194E             lea     rcx, [rbp-38h] ; alloc::string::String
*

```

```

.text:0000000140001952      call
_ZN4core3ptr42drop_in_place$LT$alloc__string__String$GT$17h14850fc5b309e955E ;
core::ptr::drop_in_place$LT$alloc..string..String$GT$::h14850fc5b309e955
.text:0000000140001957      nop
.text:0000000140001958      add     rsp, 70h
.text:000000014000195C      pop     rbp
.text:000000014000195D      retn
.text:000000014000195D ; } // starts at 140001900
.text:000000014000195D demo__test_string endp

```

上面说的那两个创建字符串的函数指向完成以后，变量s和s1中会保存字符串长度以及字符串在堆中的地址。这个通过动态调试挺容易看出，这里就不放出来了。

下面的例子，就是一个String和str之间发生隐式解引用的例子。这里func1的参数是&String，而func1的参数则是&str，和参数&s的参数类型并不一样。但是，这段代码并不会出现任何错误

```

fn test() {
    let s = String::from("Hello");

    func(&s);
    func1(&s);
}

fn func(s: &String) {

}

fn func1(s: &str) {

}

```

对应的汇编代码如下，test函数首先会将变量s的地址作为第一个参数去申请堆内存来保存字符串。期间，还会把变量s的地址保存在栈中。

```

.text:0000000140001980 demo__test      proc near
.text:0000000140001980
.text:0000000140001980                push    rbp
.text:0000000140001981                sub     rsp, 60h
.text:0000000140001985                lea     rbp, [rsp+60h]
.text:000000014000198A                mov     [rbp+var_8], 0FFFFFFFFFFFFFFFh
.text:0000000140001992                lea     rdx, aAttemptToDivid+19h ; "Hellocalled
`Option::unwrap()` on a `No"...
.text:0000000140001999                lea     rcx, [rbp-20h] ; rcx=变量s的内存地址
.text:000000014000199D                mov     [rbp-28h], rcx ; 保存变量s的内存地址
.text:00000001400019A1                mov     r8d, 5
.text:00000001400019A7                call
_ZN76_$LT$alloc__string__String$u20$as$u20$core__convert__From$LT$$RF$str$GT$$GT$4from
17hcf90cab7229f4c30E ;
_$LT$alloc..string..String$u20$as$u20$core..convert..From$LT$$RF$str$GT$$GT$::from::hc
f90cab7229f4c30

```

调用func函数的时候，会将保存在栈中的变量s的地址取出，作为第一个参数：

```

.text:00000001400019AC                mov     rcx, [rbp-28h] ; alloc::string::String
*
.text:00000001400019B0
.text:00000001400019B0 loc_1400019B0:                ; DATA XREF:
.rdata:000000014001F250↓o
.text:00000001400019B0 ; try {
.text:00000001400019B0                call    demo__func
.text:00000001400019B5                jmp     short $+2

```

而调用func1的时候，会先将变量s的地址作为第一个参数，调用_ZN65_\$LT\$alloc__string__String\$u20\$as\$u20\$core__ops__deref__Deref\$GT\$5deref17h93b928cc9020066dE函数进行解引用。随后，将解引用得到的字符串地址和长度作为参数，来调用func1函数。

```

.text:00000001400019B7 loc_1400019B7:                                ; CODE XREF:
demo__test+35↑j
.text:00000001400019B7                                lea     rcx, [rbp-20h] ; result
.text:00000001400019BB                                call
_ZN65_$LT$alloc__string__String$u20$as$u20$core__ops__deref__Deref$GT$5deref17h93b928c
c9020066dE ;
_$LT$alloc..string..String$u20$as$u20$core..ops..deref..Deref$GT$::deref::h93b928cc902
0066d
.text:00000001400019C0                                mov     [rbp-38h], rdx ; 将字符串长度保存起来
.text:00000001400019C4                                mov     [rbp-30h], rax ; 将字符串地址保存起来
.text:00000001400019C8                                jmp     short $+2
.text:00000001400019CA ; -----
-----
.text:00000001400019CA
.text:00000001400019CA loc_1400019CA:                                ; CODE XREF:
demo__test+48↑j
.text:00000001400019CA                                mov     rdx, [rbp-38h] ; rdx=字符串长度
.text:00000001400019CE                                mov     rcx, [rbp-30h] ; rcx=字符串地址
.text:00000001400019D2                                call    demo__func1
.text:00000001400019D2 ; } // starts at 1400019B0
.text:00000001400019D7
.text:00000001400019D7 loc_1400019D7:                                ; DATA XREF:
.rdata:000000014001F258↓o
.text:00000001400019D7                                jmp     short $+2

```

此时，对于func1函数的调用，和直接用&str变量作为参数时候的形式一样。因此可以看出，如果参数类型不匹配，而参数又实现了Deref trait。那么在进行函数调用的时候，编译器会自动插入解引用函数进行解引用。